

Lehrstuhl für Sensorik

Neues Gleichungsvergabekonzept in CFS++

Andreas Hauck

1 Bisherige Gleichungsnummernvergabe

Das bisherige Konzept der Gleichungsnummernvergabe beruht darauf, dass sich alle Einträge, die in das globale Gleichungssystem mit eingehen, aus Knotenunbekannten und Knotenansatzfunktionen zusammensetzen lassen. Die Zuordnung von Knoten- nach Gleichungsnummern geschieht in der Klasse `NodeEQN`. Folglich gibt es (mit Varianten) nur die zwei Befehle

```
void Node2EQN(const UInt nodeNr, const UInt dof,
               Integer & eqnNr, UInt & eqnDof) const;
void Node2EQN(const StdVector<UInt> &nodeNr,
               StdVector<Integer> &eqnNr) const;
```

um Knoten auf Gleichungsnummern zu mappen.

Aufgrund dieser Einschränkung folgende Berechnungen und Kopplungen derzeit nicht oder nur schwer umsetzbar:

- Aufstellen einer PDE mit mehreren Knotenergebnissen (z.B. Non-matching grid mit akust. Potential und Lagrange-Multipliern)
- Kopplung einer knotenbaiserten PDE mit geometrieunabhängigen Gleichungen (z.B. Kopplung ElecPDE/MagPDE + Netzwerkgleichung)
- Kopplung von Knoten- und Element-Unbekannte (z.B. Piezo-Composite Materialien)
- PDE mit Edge/Surface-Unbekannten (z.B. Magnetik in 3D)
- Verwendung von Elementen beliebiger (höherer) Ordnung unter Verwendung verschiedener Ansatzfunktionen

Ziel des neuen Klassenkonzepts muss daher eine weitergehende Abstraktion der Einträge in den SystemMatrizen und ihres Ursprungs bieten.

2 Allgemeines Konzept

In Abbildung ?? sind ist der allgemeine Weg des Mappings von einer (physikalischen) gröÙe bis zum entsprechenden Eintrag in der Matrix (definiert durch die Gleichungsnummer) gezeigt.

Abbildung 1: Ablauf des Mappings

Dabei gibt es folgende Stufen:

- **Quantity/Dof**

Am Beginn steht immer eine (meist physikalische) GröÙe, wie z.B. das elektr. Potential, die mech. Verschiebung oder der akust. Druck. Es können jedoch auch nicht-physikalische GröÙen und Unbekannte auftreten, wie z.B. die Lagrange-Multiplizierer. Jede

- **Defined On**

Jede der o.g. Größen ist entweder auf Knoten/Kanten/Oberflächen, dem Element selbst, einer Liste von Knoten (z.B. Springs) oder als freie Unbekannte (z.B. Maschenstrom usw.) definiert.

- **Interpolation/Approximation**

Abhängig von den Entities, auf denen eine Größe definiert ist, kann sie verschiedenartig approximiert werden:

- Knoten/Kanten/Oberflächen: Für diese geometrischen Elemente existieren verschiedene Ansatzfunktionen, z.B. Lagrange oder Legendre. Für diese gibt es u.U. auch verschiedene Interpolationsordnungen (linear, quadratisch, höherer Ordnung)
- Element/Region/NodeList/Free: Da diese Elemente nicht an Interpolationsfunktionen gebunden sind lassen sie sich nur als konstante Größen approximieren.

- **Entity Type**

Abhängig davon, worauf die Größe definiert ist (s. Defined On), muss über verschiedene Entities iteriert werden, um die Elementmatrizen aufzustellen. Für Knoten / Kanten / Oberflächen / Element-Unbekannte muss (wie bisher) über die Elemente einer Region iteriert werden. Für Regionen-Unbekannte muss über eine Liste von Regionen, für Unbekannte einer Knotenliste muss über diese und für unabhängige Unbekannte muss über vorher vergebene Nummern iteriert werden. Wichtig ist, dass sich jeder EntityType aufzählen lässt, sich also über ihn iterieren lässt.

- **EQN number**

Um am Ende die Einträge, die eine Größe definieren, in die Matrix eintragen zu können, ist eine Kombination aus Gleichungsnummern notwendig, um Zeile-Spalen-Kombinationen für die Matriceinträge bestimmen zu können.

Das bisherige Konzept und seine Implementierung lassen bis jetzt nur ein Mapping von Knotenunbekannten mit Lagrange-Funktionen 1./2. Ordnung zu. Das neue Konzept sollte jedoch alle o.g. Kombinationen zulassen.

3 Neue Klassenstruktur

In Abbildung ?? ist die neue Klassenstruktur zusammenhängend zu sehen.

Im Folgenden werden die neuen Klassen und ihre Rolle näher vorgestellt.

3.1 Klasse ResultDof

Diese Klasse erweitert den Enum-Type `SolutionType` so, dass auch die Information über die Art der Approximation und nähere Angaben über die DOFs enthalten sind.

Die Variable `result_` kodiert die Größe, die approximiert wird und `dofs_` die Anzahl der Freiheitsgrade. Die Namen dieser sind in `dofNames_` gespeichert.

Die Variable `definedOn_` gibt an, worauf die Größe definiert ist (NODE, EDGE, SURFACE, ELEMENT, REGION, NODLIST, FREE). Zusätzlich wird in der Variable `approxType` gespeichert, wie die Größe approximiert / interpoliert wird.

Objekte dieser Klasse werden in der Klasse `SinglePDE` angelegt (z.B. in `DefineIntegrators`) und werden den `BaseForm`-Integratoren mitgegeben, so dass sie die Interpolationsordnung wissen. Außerdem können Objekte dieses Typs für die neue `StoreSolution`-Klasse verwendet werden, da sie direkt die Größe, die dofs und deren Namen verwaltet. Das Objekt ließe sich evtl. noch um Flags für komplex/reell usw. erweitern.

- `ResultDof`
- `Approx`
- `BaseForm`
- `IntDescriptor`
- `EntityList`
- `EntityIterator`

4 Beispiele

5 Fragen / Verbesserungen

Abbildung 2: Neue Klassenstruktur

Abbildung 3: ResultDof Klasse